

A completer : Nom de l'établissement

A completer : Departement / Section

Saveur - Application de recettes de cuisine et planification de repas

Rapport academique de projet d'examen

Cours : Developpement JavaScript

Cours : Bases de donnees NoSQL

Projet : Projet 5 - Application de recettes de cuisine

Etudiants : A completer : Noms et prenomms des etudiants

Enseignant : A completer : Nom de l'enseignant

Date de remise : A completer : Date de remise

Annee academique 2025-2026

Resume

Dans le cadre des cours de Developpement JavaScript et de Bases de donnees NoSQL, nous avons realise une application web full-stack intitulee *Saveur*. L'objectif general est de permettre a un utilisateur authentifie de gerer des recettes de cuisine, de consulter un catalogue public, de filtrer les recettes par categorie, temps et difficulte, d'ajouter des recettes aux favoris, de planifier les repas de la semaine et de generer automatiquement une liste de courses a partir du planning. La solution repose sur React et TypeScript pour l'interface, Node.js et Express pour l'API REST, MongoDB et Mongoose pour la persistance, ainsi que bcrypt et JWT pour la securite. Le projet respecte une architecture separee en modele, vue et controleur : schemas Mongoose, controleurs Express, routes API et pages React. Au terme de l'implementation, l'application est fonctionnelle localement, connectee a MongoDB, et couvre les principales exigences techniques et fonctionnelles du sujet.

Table des matières

Resume	1
1 Introduction	6
2 Analyse des besoins	7
2.1 Objectifs fonctionnels	7
2.2 Objectifs non fonctionnels	7
2.3 Cibles utilisateurs	8
3 Conception technique	9
3.1 Architecture logicielle	9
3.2 Separation des responsabilites	9
3.3 Modelisation NoSQL	10
3.4 Authentification	11
4 Implementation	13
4.1 Stack technologique	13
4.2 Organisation du code	13
4.3 Fonctionnalites cles	14
4.3.1 Route API protegee	14
4.3.2 Hashage du mot de passe	14
4.3.3 Generation de la liste de courses	14
4.3.4 Appel API cote frontend	15
5 Tests et validation	16
5.1 Environnement de test	16
5.2 Plan de tests	16
5.3 Resultats	17
5.4 Difficultes rencontrees et solutions	17
5.4.1 Migration depuis un prototype mobile/Firebase	17
5.4.2 Modelisation des ingredients	17
5.4.3 Securisation des routes privees	18

5.4.4	Installation locale de MongoDB	18
6	Ameliorations possibles	19
7	Conclusion	20
A	Captures d'ecran	21
B	Guide d'installation et d'execution	25
D	Repartition des taches dans le groupe	27
	References	28

Table des figures

3.1	Architecture trois tiers de Saveur	9
3.2	Diagramme simplifie des collections NoSQL	11
3.3	Flux d'authentification	12
A.1	Tableau de bord de l'application Saveur	21
A.2	Page de connexion et d'inscription	22
A.3	Formulaire de creation d'une recette	22
A.4	Catalogue avec filtres par categorie, temps et difficulte	23
A.5	Detail d'une recette avec ingredients et etapes	23
A.6	Planning hebdomadaire des repas	24
A.7	Liste de courses generee automatiquement	24

Liste des tableaux

3.1	Collections MongoDB utilisees	10
3.2	Routes protegees et autorisations	12
5.1	Cas de test fonctionnels	16
D.1	Repartition des contributions	27

Chapitre 1

Introduction

Les cours de Developpement JavaScript et de Bases de donnees NoSQL ont pour objectif de former les etudiants a la conception d'applications web modernes, interactives et connectees a une base de donnees non relationnelle. Le projet d'examen demande la realisation d'une application web complete, composee d'une interface cliente, d'un serveur Node.js/Express, d'une base MongoDB et d'un systeme d'authentification securise.

La problematique principale consiste a transformer un besoin fonctionnel concret en une solution full-stack coherentement structuree. Pour notre groupe, le sujet retenu est l'application de recettes de cuisine. Ce domaine implique plusieurs entites liees : utilisateurs, recettes, categories, ingredients, etapes de preparation, favoris, planning de repas et liste de courses. La difficulte technique reside dans la modelisation NoSQL de ces donnees et dans la mise en place de relations par references et par embarquement.

Nous avons choisi de developper *Saveur*, une application web permettant de creer et consulter des recettes, de les organiser dans un planning hebdomadaire et de produire automatiquement une liste de courses. L'application vise une utilisation simple et directe : l'utilisateur se connecte, parcourt le catalogue, cree ses propres recettes, ajoute des favoris, planifie ses repas puis obtient les ingredients necessaires.

Ce rapport presente d'abord l'analyse des besoins, puis la conception technique de l'application. Il decrit ensuite l'implementation, l'organisation du code, les tests effectues, les difficultes rencontrees et les ameliorations possibles. Les annexes contiennent les captures d'ecran attendues, le guide d'installation et la repartition des taches.

Chapitre 2

Analyse des besoins

2.1 Objectifs fonctionnels

Les objectifs fonctionnels définissent ce que l'application doit permettre de faire. Ils proviennent du sujet de projet et des améliorations que nous avons jugées utiles pour rendre l'application cohérente.

- Permettre l'inscription et la connexion des utilisateurs.
- Sécuriser les mots de passe par hashage avant leur stockage.
- Afficher un catalogue de recettes publiques.
- Filtrer les recettes par catégorie, difficulté et temps maximal.
- Créer une recette personnelle avec titre, description, image, temps, difficulté, portions, ingrédients et étapes.
- Gérer les ingrédients avec quantités et unités.
- Ajouter ou retirer une recette des favoris.
- Consulter les recettes personnelles de l'utilisateur.
- Planifier les repas de la semaine par type de repas : petit déjeuner, déjeuner, dîner et snack.
- Générer une liste de courses intelligente à partir des recettes planifiées.
- Afficher un tableau de bord avec des indicateurs : recettes, favoris, repas planifiés et articles de courses.

2.2 Objectifs non fonctionnels

- **Sécurité** : les routes sensibles sont protégées par un token JWT, et les mots de passe sont hashés avec bcrypt.

- **Performance** : les requetes REST renvoient uniquement les donnees utiles et les listes sont limitees lorsque necessaire.
- **Ergonomie** : l'interface est organisee autour de pages claires : tableau de bord, catalogue, planning, courses et compte.
- **Maintenabilite** : le code est organise par responsabilite : modeles, controleurs, routes, middleware, pages et services.
- **Compatibilite** : l'application fonctionne dans les navigateurs modernes grace a Vite et React.
- **Reproductibilite** : le fichier `README.md` et le fichier `.env.example` permettent de reproduire l'environnement local.

2.3 Cibles utilisateurs

L'application prevoit deux types d'utilisateurs :

- **Utilisateur simple** : peut s'inscrire, se connecter, creer des recettes, consulter le catalogue, ajouter des favoris, planifier ses repas et generer sa liste de courses.
- **Administrateur** : le role existe dans le modele utilisateur. Il peut etre utilise pour des evolutions futures, par exemple la moderation des recettes ou la gestion des categories.

Chapitre 3

Conception technique

3.1 Architecture logicielle

L'application suit une architecture trois tiers. Le client React constitue la couche presentation. L'API Express constitue la couche logique metier et expose les routes REST. MongoDB constitue la couche donnees. Cette separation permet de faire evoluer l'interface sans modifier directement les schemas de base, et de tester l'API separement de la vue.

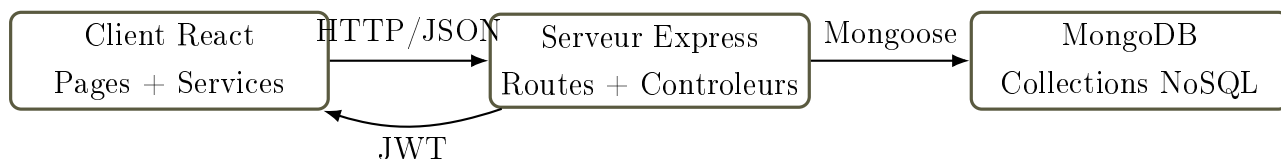


FIGURE 3.1 – Architecture trois tiers de Saveur

3.2 Separation des responsabilites

Le projet respecte le principe modele, vue, controleur sous une forme adaptee a une application React SPA.

- **Modele** : les schemas Mongoose se trouvent dans `server/models`. Ils decrivent la structure de chaque collection.
- **Controleur** : les fichiers de `server/controllers` contiennent la logique de traitement des requetes.
- **Routes** : les fichiers de `server/routes` associent les endpoints HTTP aux fonctions de controleur.
- **Vue** : les pages React dans `src/pages` affichent les donnees et gerent les interactions utilisateur.
- **Services frontend** : les fichiers `src/services` isolent les appels API afin que les vues ne manipulent pas directement `fetch`.

3.3 Modelisation NoSQL

La base utilise sept collections principales. Certaines relations sont gérées par références, et certains sous-documents sont embarqués pour optimiser la lecture.

TABLE 3.1: Collections MongoDB utilisées

Collection	Champs principaux		Relations et justification
users	name, passwordHash, timestamps	email, role,	Referencee par les recettes, favoris, plans et listes. Le mot de passe n'est jamais stocké en clair.
categories	name, image, timestamps		Referencee par recipes . La catégorie reste réutilisable pour plusieurs recettes.
ingredients	name, category, caloriesPerUnit	defaultUnit,	Referencee dans les ingrédients embarqués des recettes. Elle facilite la normalisation des noms et unités.
recipes	title, prepTime, difficulty, category, ingredients, steps	description, cookTime, servings, user,	Reference vers l'utilisateur et la catégorie. Les ingrédients et étapes sont embarqués car ils sont lus avec la recette.
mealplans	user, date, recipe	mealType,	Relie un utilisateur à une recette pour un jour et un type de repas. Index unique sur utilisateur, date et type.
favorites	user, recipe, timestamps		Collection de jonction entre utilisateurs et recettes. Index unique pour éviter les doublons.
shoppinglists	user, weekStart, items[]		Les articles sont embarqués car ils représentent le résultat calculé d'une liste hebdomadaire.

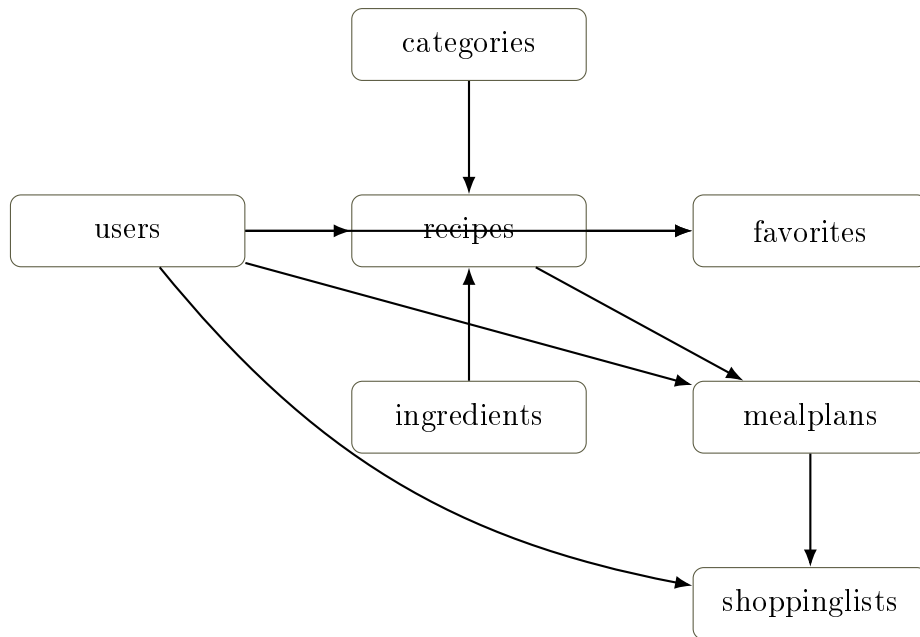


FIGURE 3.2 – Diagramme simplifié des collections NoSQL

3.4 Authentification

L'authentification utilise un mécanisme JWT. Lors de l'inscription, le mot de passe est valide puis hashé avec bcrypt. Lors de la connexion, le mot de passe fourni est comparé au hash stocké. Si les informations sont correctes, le serveur génère un token JWT contenant l'identifiant de l'utilisateur et son rôle. Ce token est ensuite envoyé par le frontend dans l'en-tête `Authorization`.

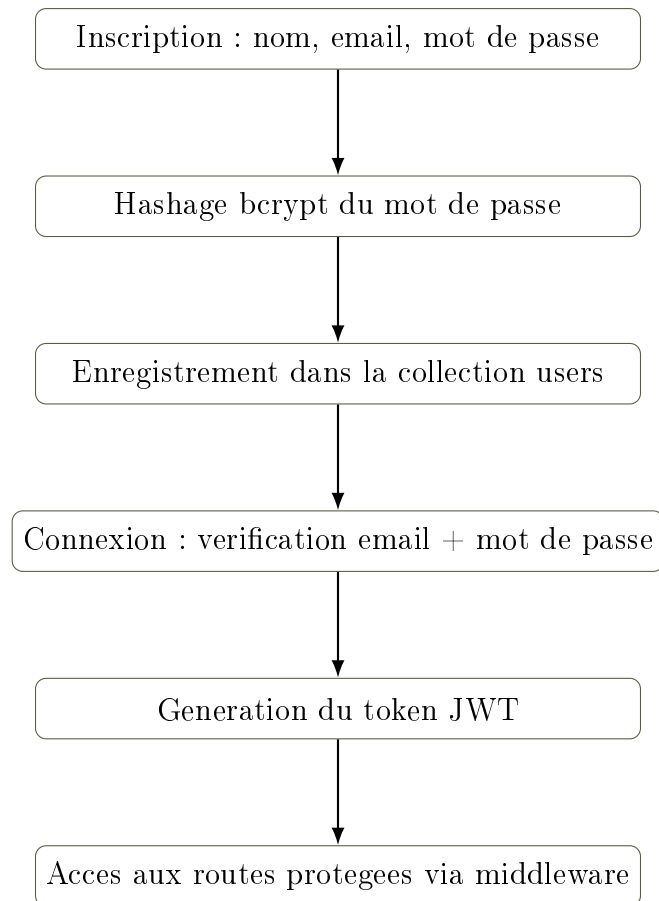


FIGURE 3.3 – Flux d’authentification

TABLE 3.2 – Routes protegees et autorisations

Route	Protection	Autorisation
GET /api/auth/me	JWT requis	Utilisateur connecte
POST /api/recipes	JWT requis	Utilisateur connecte
GET /api/recipes/mine	JWT requis	Proprietaire des recettes
DELETE /api/recipes/:id	JWT requis	Proprietaire ou administra- teur
GET /api/favorites	JWT requis	Utilisateur connecte
POST /api/favorites/:id/toggle	JWT requis	Utilisateur connecte
GET /api/meal-plans	JWT requis	Utilisateur connecte
POST /api/meal-plans	JWT requis	Utilisateur connecte
GET /api/groceries	JWT requis	Utilisateur connecte
GET /api/dashboard	JWT requis	Utilisateur connecte

Chapitre 4

Implementation

4.1 Stack technologique

- **React 19** pour l'interface utilisateur.
- **TypeScript** pour typer les données côté frontend.
- **Vite** pour le serveur de développement et la génération du build.
- **Tailwind CSS** pour le style visuel.
- **Node.js** et **Express** pour le backend REST.
- **MongoDB** comme base de données NoSQL.
- **Mongoose** pour la définition des schémas et les requêtes.
- **bcryptjs** pour le hachage des mots de passe.
- **jsonwebtoken** pour les sessions JWT.
- **lucide-react** pour les icônes de l'interface.

4.2 Organisation du code

Listing 4.1 – Arborescence simplifiée du projet

```
server/  
  config/ # connexion MongoDB  
  controllers/ # logique de traitement des requêtes  
  data/ # données de seed  
  middleware/ # authentification et erreurs  
  models/ # schémas Mongoose  
  routes/ # endpoints Express  
  utils/ # helpers JWT  
src/  
  components/ # composants partagés
```

```
pages/ # vues React
services/ # appels API frontend
types.ts # types TypeScript partages
```

4.3 Fonctionnalités clés

4.3.1 Route API protégée

La route de création de recette utilise le middleware `requireAuth`. Seul un utilisateur connecté peut créer une recette.

Listing 4.2 – Route protégée pour créer une recette

```
recipeRoutes.post('/', requireAuth, createRecipe);
```

4.3.2 Hashage du mot de passe

Lors de l'inscription, le mot de passe est transformé en hash avant insertion dans MongoDB.

Listing 4.3 – Hashage bcrypt lors de l'inscription

```
const passwordHash = await bcrypt.hash(password, 12);
const user = await User.create({
  name,
  email,
  passwordHash,
  role: 'user',
});
```

4.3.3 Génération de la liste de courses

La liste de courses regroupe les ingrédients des recettes planifiées pendant la semaine. Les quantités identiques sont additionnées par nom et unité.

Listing 4.4 – Regroupement des ingrédients de la semaine

```
for (const entry of entries) {
  for (const ingredient of entry.recipe?.ingredients || []) {
    const key = `${ingredient.name.toLowerCase()}|${ingredient.unit.toLowerCase()}';
    const current = itemMap.get(key) || {
      name: ingredient.name,
```

```

    quantity: 0,
    unit: ingredient.unit,
    category: 'Meal plan',
    checked: false,
  };

  current.quantity += Number(ingredient.quantity) || 0;
  itemMap.set(key, current);
}
}

```

4.3.4 Appel API cote frontend

Les pages React n'appellent pas directement `fetch`. Elles utilisent les services dans `src/services`, ce qui separe la vue et l'accès aux données.

Listing 4.5 – Service frontend pour filtrer les recettes

```

export const getFilteredRecipes = async (filters) => {
  const params = new URLSearchParams();
  if (filters.categoryId) params.set('categoryId', filters.categoryId);
  if (filters.difficulty) params.set('difficulty', String(filters.difficulty));
  if (filters.maxTime) params.set('maxTime', String(filters.maxTime));

  const query = params.toString() ? `?${params}` : '';
  const response = await apiFetch(`/recipes${query}`);
  return response.data;
};

```

Chapitre 5

Tests et validation

5.1 Environnement de test

Les tests ont ete effectues sur un environnement local Windows. MongoDB Server 8.3.4 fonctionne comme service local. L'API Express ecoute sur le port 5000. L'interface Vite peut etre lancee sur le port 3000 ou 3001. Les tests ont ete verifies dans un navigateur moderne et par des appels HTTP sur les endpoints principaux.

5.2 Plan de tests

TABLE 5.1: Cas de test fonctionnels

No	Cas de test	Resultat attendu	Etat
1	Inscription avec nom, email et mot de passe valide	Compte cree, mot de passe hashe, token retourne	OK
2	Connexion avec compte demo	Token JWT et donnees utilisateur retournees	OK
3	Connexion avec mauvais mot de passe	Erreur 401	OK
4	Consultation du catalogue public	Liste des recettes publiques affichee	OK
5	Filtre par categorie	Seules les recettes de la categorie sont affichees	OK
6	Filtre par difficulte	Recettes limitees au niveau choisi	OK
7	Creation d'une recette connectee	Recette creee avec ingredients et etapes	OK

No	Cas de test	Resultat attendu	Etat
8	Ajout/retrait d'un favori	L'etat favori est modifie en base	OK
9	Ajout d'une recette au planning	Repas enregistre pour une date et un type	OK
10	Generation de la liste de courses	Ingredients des recettes planifiees regroupees	OK
11	Acces a une route protegee sans token	Erreur d'authentification	OK
12	Build de production	Generation du dossier <code>dist</code> sans erreur	OK

5.3 Resultats

Les tests essentiels valident le fonctionnement de la chaine complete : frontend, API REST, authentication et base MongoDB. Les commandes `npm run lint` et `npm run build` ont ete executees avec succes. Le backend retourne egalement un statut positif sur `/api/health`. Le compte demo `chef@saveur.local` permet de verifier rapidement le parcours utilisateur.

5.4 Difficultes rencontrees et solutions

5.4.1 Migration depuis un prototype mobile/Firebase

Le projet initial contenait des fichiers Android et Firebase. Or, le sujet exigeait une application web JavaScript avec Node.js, Express et MongoDB. Nous avons donc supprime les fichiers non pertinents et remplace la logique Firestore par une API REST connectee a MongoDB.

5.4.2 Modelisation des ingredients

La generation d'une liste de courses exige de connaitre les quantites et les unites. Une simple liste de noms d'ingredients aurait ete insuffisante. La solution retenue consiste a embarquer dans chaque recette les ingredients avec `name`, `quantity`, `unit` et `optional`, tout en gardant une collection `ingredients` reutilisable.

5.4.3 Sécurisation des routes privées

Les routes telles que les favoris, les recettes personnelles, le planning et les courses doivent appartenir à l'utilisateur connecté. Nous avons ajouté un middleware `requireAuth` qui vérifie le JWT et charge l'utilisateur avant de continuer la requête.

5.4.4 Installation locale de MongoDB

L'installation de MongoDB Server a d'abord échoué à cause d'un manque d'espace disque. Après libération d'espace, l'installation a réussi et le service Windows MongoDB a été démarré. L'application utilise ensuite l'URI locale `mongodb://127.0.0.1:27017/saveur`.

Chapitre 6

Améliorations possibles

- Ajouter une interface administrateur complète pour gérer les catégories, modérer les recettes et consulter les statistiques globales.
- Ajouter l'édition complète des recettes existantes, avec modification des ingrédients et des étapes.
- Déployer l'application sur une plateforme cloud avec une base MongoDB Atlas.
- Ajouter des tests automatisés backend et frontend.
- Exporter la liste de courses en PDF ou en fichier imprimable.
- Ajouter des informations nutritionnelles plus détaillées.

Chapitre 7

Conclusion

Le projet *Saveur* nous a permis de mettre en pratique les competences attendues dans les deux cours. Nous avons construit une application web full-stack avec une interface dynamique, une API REST, une base MongoDB et un systeme d'authentification securise. Les objectifs principaux du sujet sont atteints : gestion des recettes, catalogue avec filtres, favoris, recettes personnelles, tableau de bord, planification de repas et generation de liste de courses.

Sur le plan technique, le projet nous a permis de mieux comprendre la separation des responsabilites, la modelisation NoSQL et la securite cote serveur. Nous avons egalement appris a adapter une application existante a un cahier des charges different, en retirant les parties non conformes et en reconstruisant une architecture plus appropriee.

Comme perspective, l'application peut encore etre enrichie par un espace administrateur, un deploiement en ligne, des tests automatises et des exports PDF. Neanmoins, la version actuelle constitue une base fonctionnelle et conforme aux exigences principales de l'examen.

Annexe A

Captures d'écran

Les figures suivantes doivent être remplacées par les captures finales avant la remise. Le modèle demande au moins six captures : page d'accueil, connexion, formulaire d'ajout, liste avec filtres, dashboard/graphiques et fonctionnalité améliorée.

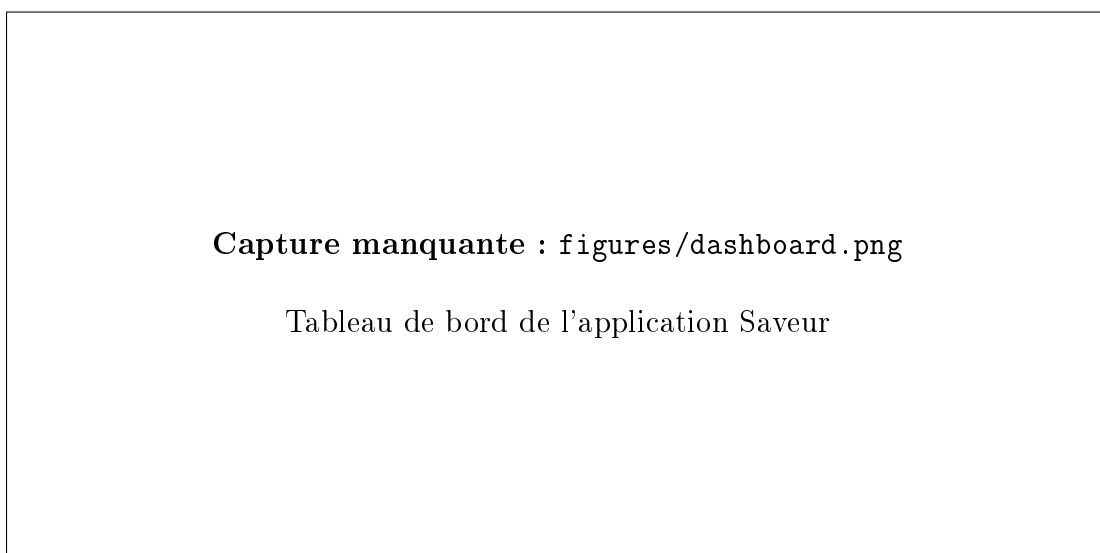


FIGURE A.1 – Tableau de bord de l'application Saveur

Capture manquante : figures/login.png

Page de connexion et d'inscription

FIGURE A.2 – Page de connexion et d'inscription

Capture manquante : figures/create-recipe.png

Formulaire de creation d'une recette

FIGURE A.3 – Formulaire de creation d'une recette

Capture manquante : figures/catalog-filters.png

Catalogue avec filtres par categorie, temps et difficulte

FIGURE A.4 – Catalogue avec filtres par categorie, temps et difficulte

Capture manquante : figures/recipe-detail.png

Detail d'une recette avec ingredients et etapes

FIGURE A.5 – Detail d'une recette avec ingredients et etapes

Capture manquante : figures/meal-planner.png

Planning hebdomadaire des repas

FIGURE A.6 – Planning hebdomadaire des repas

Capture manquante : figures/grocery-list.png

Liste de courses generee automatiquement

FIGURE A.7 – Liste de courses generee automatiquement

Annexe B

Guide d'installation et d'exécution

Prerequis

- Node.js installé.
- MongoDB Server installé et démarre.
- Un terminal dans le dossier du projet.

Etapes

1. Installer les dependances :

```
npm install
```

2. Créer le fichier d'environnement :

```
cp .env.example .env
```

Sur Windows PowerShell :

```
Copy-Item .env.example .env
```

3. Vérifier que MONGO_URI vaut :

```
mongodb://127.0.0.1:27017/saveur
```

4. Démarrer l'API :

```
npm run api
```

5. Démarrer le frontend dans un autre terminal :

```
npm run dev
```

6. Ouvrir le navigateur sur :

```
http://localhost:3000
```

Compte demo

— Email : `chef@saveur.local`

— Mot de passe : `saveur123`

Annexe D

Repartition des taches dans le groupe

TABLE D.1 – Repartition des contributions

Nom et prenom	Contribution
A completer	Analyse du sujet, modelisation NoSQL, backend Express
A completer	Interface React, pages et services frontend
A completer	Tests, documentation, captures d'ecran et rapport
A completer	Integration, correction des erreurs et preparation GitHub

Bibliographie

- [1] Documentation officielle Node.js, <https://nodejs.org/>.
- [2] Documentation officielle Express, <https://expressjs.com/>.
- [3] Documentation officielle MongoDB, <https://www.mongodb.com/docs/>.
- [4] Documentation officielle Mongoose, <https://mongoosejs.com/>.
- [5] Documentation officielle React, <https://react.dev/>.
- [6] Documentation officielle Vite, <https://vite.dev/>.